

DCPO: DYNAMIC CLIPPING POLICY OPTIMIZATION

Anonymous authors
Anonymous submission

ABSTRACT

Reinforcement Learning from Verifiable Rewards (RLVR) has emerged as a promising framework for enhancing the reasoning capabilities of large language models, yet existing approaches such as GRPO often waste gradient signal: fixed clipping bounds on token-level probability ratios and the standardization of identical rewards jointly produce zero gradients, ineffective updates, and underutilization of generated responses. This paper makes three contributions that directly target these failure modes. **First**, we propose **Dynamic Clipping Policy Optimization (DCPO)**, a unified RLVR pipeline built from two novel and complementary mechanisms: (i) a *dynamic-adaptive clipping* strategy that widens clipping bounds for tokens with low prior probability to unlock token-level exploration, and (ii) a *smooth advantage standardization* technique that aggregates rewards across cumulative training steps to raise the response-level utilization of generated data. **Second**, we show that these mechanisms translate into state-of-the-art accuracy: across four benchmarks and four different models, DCPO consistently leads. In particular, on the Qwen2.5-Math-7B model DCPO reaches an Avg@1 of 46.7 under greedy decoding and an Avg@32 of 38.8 under 32-times sampling on AIME24, surpassing DAPO (36.7/31.6), GRPO (36.7/32.1) and GSPO (40.0/34.9); on AIME25 with Qwen2.5-14B it attains (23.3/19.0), surpassing GRPO (13.3/10.5), DAPO (20.0/15.3) and GSPO (16.7/9.9). **Third**, we demonstrate that these accuracy gains come with markedly better data efficiency: DCPO improves the nonzero-advantage ratio over GRPO by 28% on average across the four models, doubles training efficiency relative to DAPO, and reduces the token clipping ratio by an order of magnitude compared to both GRPO and DAPO. Together, these results establish DCPO as an effective and data-efficient framework for reinforcement learning in large language models.

1 INTRODUCTION

In recent years, the reasoning capabilities of large language models (LLMs) have garnered increasing attention (OpenAI, 2025; Guo et al., 2025; Comanici et al., 2025). Reinforcement Learning from Verifiable Rewards (RLVR) represents a promising technique for enhancing these capabilities: it leverages rule-based outcome rewards to optimize LLMs and is especially effective in domains that demand reflective decision-making, such as mathematics and coding (DeepSeek-AI et al., 2024). This paper asks how to make RLVR use its generated data more fully, and answers with a single pipeline whose contributions we preview below.

To support RLVR, the GRPO (DeepSeek-AI et al., 2024; Liu et al., 2025; Yuan et al., 2025; Zheng et al., 2025) family of algorithms was developed. Although GRPO’s robustness has been extensively validated, recent work such as DAPO (Yu et al., 2025) exposes two critical limitations that motivate our contributions. First, GRPO suffers from entropy collapse, in which advantage values decay to zero in later training stages. Second, inappropriate weight-clipping discards rarely sampled learning signals, over-restricting the diversity of model exploration.

Existing remedies only partially close these gaps. DAPO, for instance, introduces the Clip-Higher strategy to relax the upper clipping bound against entropy collapse, together with a Dynamic Sampling strategy that filters out samples with uniform rewards so that every batch retains valid gradients. Yet these methods leave two problems unresolved, and it is precisely these two problems that our contributions target: (i) **Response-Level Inefficiency**: dynamic sampling discards responses and thus lowers sampling efficiency, slowing convergence; and (ii) **Token-Level Clipping Deficit**: our

analysis shows that the optimal clipping bound varies across tokens, so any uniform upper bound is suboptimal. Even methods that avoid directly discarding zero-gradient responses inherit these limitations.

Our approach. To resolve both problems within one framework, we introduce **Dynamic Clipping Policy Optimization (DCPO)**. At a high level, DCPO tackles the token-level clipping deficit with a probability-aware clipping rule and the response-level inefficiency with a cumulative advantage-smoothing rule, and it further corrects a loss-averaging bias that dilutes per-response signal. Concretely, DCPO contributes:

- **Contribution 1 — Dynamic-Adaptive Clipping (DAC).** We replace fixed clipping bounds with bounds that adapt to each token’s old (prior) probability, granting tokens with lower prior probability wider bounds and thus more room for token-level exploration (Section 2 and Appendix A.6).
- **Contribution 2 — Smooth Advantage Standardization (SAS).** We standardize advantages across *cumulative* training steps rather than a single step, aggregating the reward distribution of all responses generated so far. This prevents the zeroing-out of advantages when identically rewarded responses occur, enabling more efficient response-level exploration (Section 2).
- **Contribution 3 — Only-Token-Mean loss (OTM).** We average the loss over tokens *within* each response rather than over the whole batch, preserving the relative advantage structure among responses to the same prompt and avoiding the dilution effect of batch-level averaging.
- **Contribution 4 — Empirical validation.** On four models and four mathematical reasoning benchmarks, DCPO delivers state-of-the-art accuracy *and* substantially better data efficiency than GRPO, DAPO, and GSPO (Section 5).

We validate these contributions on a training set that merges Math-DAPO-17k (Yu et al., 2025) with levels 3–5 of the MATH dataset (Liu et al., 2025), using GRPO, DAPO and (where available) GSPO as baselines on four common mathematical reasoning benchmarks. The evidence directly supports the claims above: DCPO consistently wins, with especially large gains on the challenging AIME benchmarks — on AIME24-Avg@32 (32-time sampling), DCPO-7B scores 38.8 vs. 32.1 (GRPO) vs. 31.6 (DAPO) vs. 34.9 (GSPO), and on AIME25-Avg@32, DCPO-14B reaches 19.0 vs. 10.5 (GRPO) vs. 15.3 (DAPO) vs. 9.9 (GSPO). On the efficiency side, DCPO raises the average nonzero-advantage ratio by 28% over GRPO, doubles training efficiency over DAPO, and reduces token clipping by an order of magnitude relative to all baselines — all while delivering superior accuracy.

For preliminary details, including loss and advantage calculation methods for previous algorithms such as GRPO, DAPO and GSPO, see Appendix A.2.

2 DYNAMIC-ADAPTIVE CLIPPING BOUNDS FOR RL(DAC)

This section develops our **first contribution**, Dynamic-Adaptive Clipping (DAC), which replaces fixed clipping bounds with probability-aware bounds. We begin from why clipping is needed at all. Clipping the probability ratio is a widely adopted technique in reinforcement learning to stabilize policy updates and prevent large, destabilizing parameter shifts during optimization, and it is particularly important when applying importance sampling in off-policy settings. When importance sampling is reweighted by the ratio $r(x)$ between the new probability $p(x)$ and the old probability $q(x)$, an unbiased estimate of $\mathbb{E}_{q(x)}[f(x)\frac{p(x)}{q(x)}]$ can be obtained, but the variance of such an estimate can be significantly inflated, as described in Equation (1).

$$\text{Var}_{x \sim q} \left[f(x) \frac{p(x)}{q(x)} \right] - \text{Var}_{x \sim p} [f(x)] = \mathbb{E}_{x \sim p} \left[f(x)^2 \left(\frac{p(x)}{q(x)} - 1 \right) \right] = \int f(x)^2 \left(\frac{p(x)}{q(x)} - 1 \right) p(x) dx \quad (1)$$

To mitigate this variance inflation, prior works(e.g., Schulman et al. (2017)) constrain the probability ratio within a fixed clipping bound by hyperparameter ϵ as Equation (2).

$$|r(x) - 1| \leq \epsilon \quad (2)$$

The key limitation we address is this: a symmetric fixed clipping bound constrains the model’s exploration of the diversity space, particularly in regions where the old policy assigns low probability mass. In these scenarios, the absolute bound for decreasing probabilities is inherently limited, restricting the capacity for meaningful policy updates. Although methods like DAPO attempt to

address this issue with asymmetric clipping bounds, the fundamental limitation persists. Furthermore, GSPO discards entire responses when the sequence-level importance ratio exceeds the asymmetric clipping bounds, wasting more informative tokens and retaining more high-variance tokens, which leads to training instability.

Our contribution starts from a variance-bias perspective: rather than a constant bound, we impose a *probability-dependent* constraint, as formalized in Equation (3).

$$|(r(x) - 1)p(x)| \leq \epsilon \quad (3)$$

Substituting $p(x) = r(x)q(x)$ into Equation (3) and ensuring non-negativity of the square root term by taking its maximum with zero, we obtain a closed-form solution for $r(x)$. This solution is the core of our first contribution: **dynamic-adaptive clipping bounds** that vary adaptively with $q(x)$ and the hyperparameters ϵ_{low} and ϵ_{high} , as specified in Equation (4).

$$0.5 + \frac{1}{2} \sqrt{\max\left(1 - \frac{4\epsilon_{low}}{q(x)}, 0\right)} \leq r(x) \leq 0.5 + \frac{1}{2} \sqrt{1 + \frac{4\epsilon_{high}}{q(x)}} \quad (4)$$

The payoff of this first contribution is direct: the dynamic-adaptive clipping method allows greater policy exploration exactly in the low-probability regions where fixed clipping is most restrictive. To keep this added freedom safe, and inspired by the dual clipping method (Ye et al., 2020), we cap the maximum bounds for both positive and negative advantages at 10, preventing excessive clipping bounds from overtraining. For the full derivation and analysis of these dynamic clipping bounds from the importance sampling framework, see Appendix A.5 and Appendix A.6.

3 SMOOTH ADVANTAGE STANDARDIZATION(SAS)

This section develops our **second contribution**, Smooth Advantage Standardization (SAS), which targets the response-level inefficiency identified in the introduction. To motivate it, we first isolate why single-step standardization wastes data. In previous works, such as GRPO, DAPO and GSPO, the advantage $\hat{A}_{new,j}^i$ of the j -th response is the standardized result of only taking the rewards $R_{j=1,\dots,G}^i$ for the same prompt at the i -th step, as in Equation (11) in Appendix A.3. This single-step scheme creates two problems that SAS is designed to fix. First, when randomness in response sampling makes all rewards identical at a given step, the advantage collapses to zero, so the prompt contributes nothing to parameter updates despite potentially valuable differences in reasoning trajectories. Second, randomness in high-entropy sampling can yield highly skewed label counts, causing large fluctuations in standardized advantage values across steps — even sign reversals — which destabilizes training.

Our design rests on an observation from recent work (Yue et al., 2025): RLVR primarily refines a policy to adjust the likelihood of correct samples rather than fundamentally altering the existing distribution. If so, the reward distribution of responses to the same prompt can be treated as draws from a single global distribution across training. This is the assumption underlying SAS, and it motivates a *cumulative* standardization, given in Equation (5).

$$\hat{A}_{total,j}^i = \frac{(R_j^i - \mu_{total}^i)}{\sigma_{total}^i} \quad (5)$$

where the statistics are computed over all responses generated so far for the same prompt.

Neither pure standardization is enough on its own, so the heart of this contribution is to blend them. To mitigate the fluctuations of both step-specific standardization ($\hat{A}_{new,j}^i$) and cumulative standardization ($\hat{A}_{total,j}^i$), we introduce two smoothing functions, $\hat{S}A_{new,j}^i$ and $\hat{S}A_{total,j}^i$, each a weighted average of the two standardizations with weights that shift over step i , as defined in Equation (6).

$$\hat{S}A_{new,j}^i = \frac{i-1}{i} \hat{A}_{new,j}^i + \frac{1}{i} \hat{A}_{total,j}^i, \quad \hat{S}A_{total,j}^i = \frac{1}{i} \hat{A}_{new,j}^i + \frac{i-1}{i} \hat{A}_{total,j}^i \quad (6)$$

The two terms play complementary roles: $\hat{S}A_{new,j}^i$ places more weight on the current distribution as training progresses, favoring adaptation to the latest policy outputs, whereas $\hat{S}A_{total,j}^i$ leans on the cumulative distribution, treating the current step’s deviation as a fine-tuning adjustment within the broader distribution. To turn this pair into a single stable signal — the concrete output of our second contribution — we define the final advantage $\hat{A}_j^i$ as whichever smoothed value has the smaller absolute magnitude, damping the fluctuations of both standardizations, as in Equation (7).

$$\hat{A}_j^i = \begin{cases} \hat{S}A_{new,j}^i, & \text{when } |\hat{S}A_{new,j}^i| < |\hat{S}A_{total,j}^i| \\ \hat{S}A_{total,j}^i, & \text{otherwise} \end{cases} \quad (7)$$

This design directly delivers the response-level efficiency promised in the introduction: once a prompt enters optimization, its responses in later steps keep contributing regardless of whether the current-step advantage is zero. In particular, when identical rewards occur within a step, these responses still update the model through the term $\frac{1}{i} \hat{A}_{total,j}^i$, preserving useful learning signals and improving data efficiency.

3.1 OTM LOSS

This subsection develops our **third contribution**, the Only-Token-Mean (OTM) loss, which corrects an averaging bias that neither of the two loss conventions avoids. Consider first the **Token-Level Mean loss (TLM)** (the loss in Equation (13) used by DAPO): although it balances all token contributions, it disrupts the relative relationship between responses of different lengths. For instance, take two responses to the same prompt — response A with advantage 1 and length 500, and response B with advantage 0.5 and length 1500. Under TLM, A is weighted by $\frac{500}{500+1500} = 0.25$ and B by $\frac{1500}{500+1500} = 0.75$, so their contributions become $1 \times 0.25 = 0.25$ and $0.5 \times 0.75 = 0.375$. The result is perverse: B exerts the larger influence (0.375) despite its lower advantage (0.5).

The alternative fares no better. Averaging across the batch via the **Sequence-Level Mean loss (SLM)** (the loss in Equation (12) used by GRPO) weakens the advantage magnitude, effectively shrinking it by a factor of G ; with a large batch this destroys the correlations encoded in the standardized advantages. Our contribution is the observation — and the fix — that standardized advantages *already* carry the correct relative relationships across responses, so averaging over the $|o_i|$ tokens of a single response both preserves that relationship and spreads importance equally over that response’s tokens. We therefore average only within each response and never across responses in the same batch, yielding the **Only Token Mean loss (OTM)** in Equation (8).

$$\mathcal{T}_{\text{DCPO}}(\theta) = \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \min \left(r_{i,t}(\theta) \hat{A}_{i,t}, \text{clip}(r_{i,t}(\theta), 1 - \varepsilon_{\text{low}}, 1 + \varepsilon_{\text{high}}) \hat{A}_{i,t} \right) \quad (8)$$

Two design choices tie this loss back to our other contributions. As in DAPO, the DCPO loss is not constrained by the KL divergence, and its lower and upper clipping bounds are exactly the dynamic-adaptive bounds of our first contribution (Section 2). Together with SAS, OTM completes the DCPO pipeline: dynamic clipping frees token-level exploration, cumulative smoothing recovers wasted responses, and per-response averaging preserves the advantage structure that both rely on.

4 EXPERIMENTAL SETUP

Models and Datasets To test whether our contributions generalize across model families and scales, we experimented with four different models: Qwen2.5-Math-1.5B-Instruct, Qwen2.5-3B (common base), Qwen2.5-Math-7B (math base) and Qwen2.5-14B (common base) (Yang et al., 2024a;b). We used a merged training set of roughly 25k mathematical problems that combines the DAPO-Math-17K corpus (Yu et al., 2025) with levels 3–5 of the MATH subdataset (Hendrycks et al., 2021b; Liu et al., 2025). All problems were processed with the Qwen-Math template (see Section A.8) for both training and evaluation.

Evaluation We evaluated on four widely used mathematical reasoning benchmarks: MATH500 (Hendrycks et al., 2021a), AMC23 (Ouyang et al., 2022), AIME24 (Li et al., 2024), and

AIME25 (MAA). To make the accuracy claim behind our fourth contribution robust, we use two metrics. For **MATH500** we report only Avg@1, given its larger size (500 problems). For the smaller **AMC23** (40 problems), **AIME24** (30 problems), and **AIME25** (30 problems) datasets, we report both Avg@1 and Avg@32 for a comprehensive picture.

- **Avg@1**: standard accuracy under greedy decoding, capturing the model’s best single prediction.
- **Avg@32**: the average accuracy over 32 sampled responses per problem (temperature 1.0, top-p 1.0), capturing the robustness and stability of the trained policy distribution.

Baseline Settings To situate our contributions against the strongest available alternatives, we used GRPO and DAPO as baselines throughout and added GSPO on Qwen2.5-Math-7B and Qwen2.5-14B. Remaining settings — the various clipping thresholds ϵ_* , the codebase, and other shared training hyperparameters — are described in Appendix A.9.

5 RESULTS AND ANALYSIS

5.1 MAIN RESULTS

Table 1: Main-result evidence for our accuracy contribution: performance across benchmarks. Avg@1: greedy decoding; Avg@32: 32-times sampling. Boldface marks the best value in each column, and Δ reports DCPO’s improvement over each baseline.

Model	MATH500 (Avg@1)	AMC23 (Avg@1/32)	AIME24 (Avg@1/32)	AIME25 (Avg@1/32)	Average (Avg@1/32)
Qwen2.5-Math-1.5B-Instruct					
base	73.6	57.5/49.4	10.0/10.0	3.3/6.1	36.1/21.8
GRPO	77.2	70.0/68.4	16.7/14.0	20.0/13.5	46.0/32.0
DAPO	76.0	80.0/70.6	20.0/13.5	14.4/12.5	46.5/32.4
DCPO	77.2	75.0/ 70.8	20.0/15.6	16.7/12.1	47.2/32.8
Δ_{GRPO}	+0.0	+5.0/+2.4	+3.3/+1.6	-3.3/-1.4	+1.2/+0.8
Δ_{DAPO}	+1.2	-5.0/ +0.2	+0.0/+2.1	+2.3/-0.4	+0.7/+0.4
Qwen2.5-3B[Base]					
base	46.4	27.5/7.8	3.3/0.1	3.3/0.7	20.1/2.9
GRPO	69.2	62.5/51.6	10.0/7.5	6.7/4.5	36.3/21.0
DAPO	72.4	57.5/54.0	10.0/8.3	6.7/3.9	36.6/23.1
DCPO	71.2	62.5/55.8	3.3/7.5	10.0/4.7	37.6/22.7
Δ_{GRPO}	+2.0	+0.0/+4.2	-6.7/ +0.0	+3.3/+0.2	+1.3/+1.7
Δ_{DAPO}	-1.2	+5.0/+1.8	-6.7/-0.8	+3.3/+0.8	+1.0/-0.4
Qwen2.5-Math-7B[Base]					
base	50.4	40.0/19.5	13.3/6.0	3.3/1.5	28.4/9.3
GRPO	81.6	77.5/75.9	36.7/32.1	16.7/16.7	53.1/41.6
DAPO	83.0	72.5/ 80.7	36.7/31.6	23.3/14.9	53.9/42.4
GSPO	84.0	80.0/78.8	40.0/34.9	16.7/16.2	55.2/43.3
DCPO	82.5	82.6/79.8	46.7/38.8	16.7/ 17.2	57.1/45.2
Δ_{GRPO}	+0.9	+5.1/+4.9	+10.0/+6.7	+0.0/+0.5	+4.0/+3.6
Δ_{DAPO}	-0.5	+10.1/-0.9	+10.0/+7.2	-6.6/ +2.3	+3.3/+2.8
Δ_{GSPO}	-1.5	+2.6/+1.0	+6.7/+3.9	+0.0/+1.0	+1.9/+1.9
Qwen2.5-14B[Base]					
base	60.8	47.5/16.4	3.3/1.3	3.3/1.1	28.7/6.3
GRPO	81.2	75.0/65.6	13.3/17.6	13.3/10.5	45.7/31.3
DAPO	83.4	87.5/85.1	16.7/16.4	20.0/15.3	51.9/38.9
GSPO	78.6	77.5/75.0	23.3/16.0	16.7/9.9	49.0/33.5
DCPO	84.6	85.0/79.9	20.0/ 18.2	23.3/19.0	53.2/39.0
Δ_{GRPO}	+3.4	+10.0/+14.3	+6.7/+0.6	+10.0/+8.5	+6.5/+7.7
Δ_{DAPO}	+1.2	-2.5/-5.2	+3.3/+1.8	+3.3/+3.7	+1.3/+0.1
Δ_{GSPO}	+6.0	+7.5/4.9	-3.3/ +2.2	+6.6/+10.1	+4.2/+5.5

We now present the evidence for our fourth contribution — that DCPO’s mechanisms yield state-of-the-art accuracy. Table 1 compares GRPO, DAPO, GSPO, and DCPO on MATH500, AMC23, AIME24, and AIME25, reporting both greedy decoding (Avg@1) and sampling-based decoding (Avg@32) to gauge robustness and generalization. The headline finding is consistency: across all four model scales, DCPO matches or surpasses GRPO, DAPO and (where available) GSPO, confirming it as a general-purpose preference optimization technique for RLVR rather than a method tuned to a single setting.

Result 1: Overall Superiority of DCPO The average scores establish the core claim, one model at a time. On Qwen2.5-Math-1.5B-Instruct, DCPO tops the table at (47.2/32.8), exceeding GRPO (+1.2/+0.8) and DAPO (+0.7/+0.4). On Qwen2.5-3B, DCPO reaches (37.6/22.7), beating GRPO (+1.3/+1.7) and remaining competitive with DAPO — ahead on Avg@1 (+1.0) with only a marginal Avg@32 gap (-0.4). On Qwen2.5-Math-7B, DCPO attains (57.1/45.2), surpassing GRPO (+4.0/+3.6), DAPO (+3.3/+2.8), and GSPO (+1.9/+1.9). On the largest model, Qwen2.5-14B, DCPO delivers (53.2/39.0), improving substantially over GRPO (+6.5/+7.7) and outperforming DAPO (+1.3/+0.1) and GSPO (+4.2/+5.5) in Avg@1/Avg@32. Taken together, these numbers show that DCPO’s gains are robust across model capacities under both greedy and stochastic decoding — the generality our fourth contribution claims.

Result 2: Robustness on AIME under Sampling (Avg@32) The exploration benefit of our clipping and smoothing contributions is clearest on the hardest benchmarks. AIME24 and AIME25 are high-difficulty competition math sets that stress-test reasoning, and Avg@32 measures whether a model can generate correct responses under high-entropy sampling. Here DCPO’s robustness stands out at the 7B and 14B scales: on AIME24-Avg@32, DCPO-7B reaches 38.8, outperforming GRPO (32.1), DAPO (31.6), and GSPO (34.9); on AIME25-Avg@32, DCPO-14B reaches 19.0, exceeding GRPO (10.5), DAPO (15.3), and GSPO (9.9). This is exactly the signature we would expect if DCPO enriches the diversity and reliability of correct generations, which matters most under sampling-based decoding with temperature-driven variability. In short, the ability to optimize both reward fidelity and exploration under high-entropy decoding is what gives DCPO its practical edge when many plausible outputs are sampled.

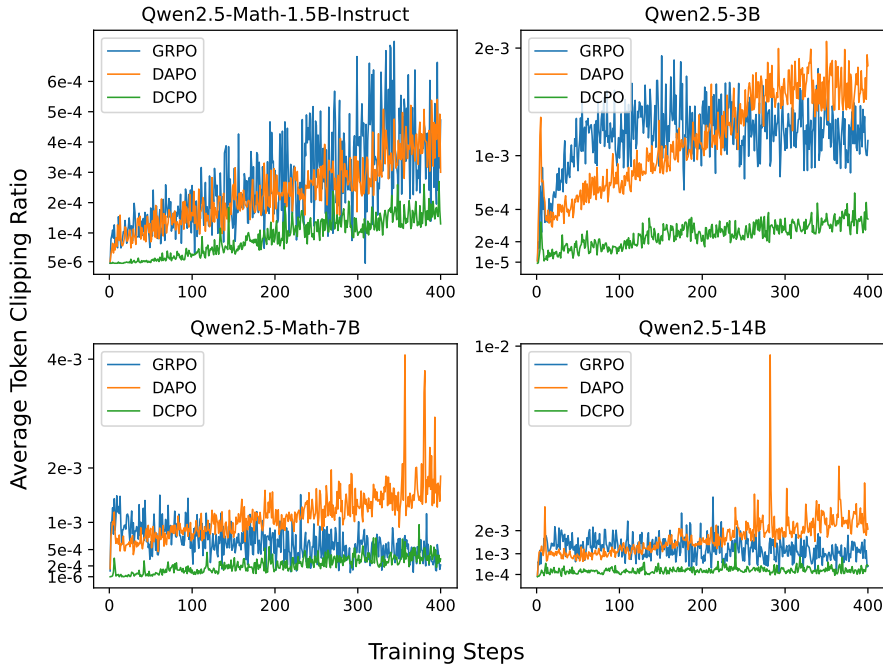


Figure 1: Token Clipping Ratio (TCR) across models and methods — evidence that our dynamic-adaptive clipping contribution discards far fewer tokens.

5.2 THE TOKEN CLIPPING RATIO (TCR)

We now quantify the token-level efficiency behind our first contribution. We define the **Token Clipping Ratio** (TCR) as the proportion of tokens excluded from policy updates during back-propagation due to clipping; a lower TCR means more tokens survive to inform the update. As Figure 1 shows, TCR behaves very differently across reinforcement learning methods.

$$TCR = Average \left(\sum_{m=1}^N \frac{\text{Number of clipped tokens in } micro_m}{\text{Total number of tokens in } micro_m} \right) \quad (9)$$

where $micro_m$ denotes the m -th microbatch and N is the number of microbatches per training step.

The baselines all pay a clipping cost that DCPO avoids. Under GRPO, TCR trends diverge with model size: it rises with training steps for smaller models (Qwen2.5-Math-1.5B and Qwen2.5-3B) but gradually falls for larger ones (Qwen2.5-Math-7B and Qwen2.5-14B). DAPO instead shows a consistently upward TCR for every model, meaning that as training proceeds more tokens are clipped and more updates rest on partial or truncated responses. Worse, both GRPO and DAPO exhibit large fluctuations and occasional abnormal spikes in TCR , which can inject instability and trigger entropy collapse. GSPO fares even worse at the sequence level: its average TCR exceeds 11% on Qwen2.5-Math-7B and 15% on Qwen2.5-14B, far above token-level methods, wasting many informative tokens.

Against this backdrop, DCPO’s behavior is the direct signature of our contribution: it stays remarkably stable and efficient. Regardless of model size or training stage, DCPO’s TCR remains nearly constant and an order of magnitude lower than GRPO’s and DAPO’s. This low, stable TCR means DCPO discards fewer tokens, lets a larger share of complete response sequences reach the policy update, and frees more room to explore rare tokens — confirming the superior sample efficiency our dynamic-adaptive clipping was designed to deliver while tightly bounding outlier variation.

To explain why this low TCR arises, take Qwen2.5-Math-7B as an example: the token distribution quickly concentrates on high-confidence tokens — about 95% of generated tokens satisfy ($q(x) > 0.9$) after roughly 60 steps, rising to about 97% after roughly 100 steps and increasing further later in training. As illustrated in Figure 4d, once the old probability satisfies ($q(x) \geq \frac{1}{1+\epsilon_{grpo}} \approx 0.83$), the new probability $p(x)$ participates in updates within the interval $[\frac{1}{1+\epsilon}, 1]$ without being discarded under any clipping method. The vast majority of tokens therefore lie in a never-clipped region, which explains the low overall TCR . The flip side, visible in Figure 1, is that baseline TCR grows in later training phases, so an increasing fraction of low-probability tokens is discarded — precisely restricting the exploration of rare tokens that reasoning depends on.

This is where our first contribution matters. Prior work (Wang et al., 2025) shows that **high-entropy tokens** — exactly the low-probability tokens — are the primary drivers of emergent reasoning. Since entropy and probability are inversely related, rarer tokens carry more information, so widening their admissible update range enlarges the effective exploration space and helps acquire diverse reasoning skills.

These empirical and theoretical findings justify our **dynamic-adaptive clipping** scheme, which relaxes clipping for tokens with $q(x) < \frac{1}{1+\epsilon}$ while keeping the safe interval for high-confidence tokens intact. By selectively expanding the update interval for the high-entropy tail, DCPO restores the model’s ability to learn from rare, informative tokens and thereby strengthens reasoning — closing the loop between the mechanism proposed in Section 2 and the TCR evidence reported here.

5.3 THE RESPONSE UTILIZATION RATIO (RUR)

We now quantify the response-level efficiency behind our second contribution. During RLVR training, a response contributes to a policy update only if it has a nonzero gradient, so we measure the **Response Utilization Ratio** (RUR): the percentage of generated responses with nonzero advantage that participate in the policy update. A higher RUR means SAS is recovering signal that single-step standardization would have thrown away.

$$RUR = \frac{\text{Number of responses with non-zero advantage}}{\text{Total number of generated responses}} \times 100\% \quad (10)$$

Table 2 makes the gap concrete. GRPO utilizes responses poorly, averaging only 44.6% across the four models — more than half of every batch of generated responses is discarded, a striking inefficiency during diversity exploration. DCPO, by contrast, reaches an average *RUR* of about 70% after the first epoch: an absolute gain of 28% and a relative gain of 64% over GRPO. This is the direct, quantified payoff of our smooth advantage standardization contribution.

Table 2: Average *RUR* over 400 training steps: DCPO nearly doubles response utilization over GRPO, evidencing our second contribution.

model	GRPO	GSPO	DCPO
Qwen2.5-Math-1.5B-Instruct	45.6%	-	67.1%
Qwen2.5-3B	48.3%	-	74.3%
Qwen2.5-Math-7B	37.4%	43.5%	73.2%
Qwen2.5-14B	43.9%	47.6%	72.4%
Average	43.8%	45.6%	71.8%

The training-time dynamics tell the same story. Figure 2 shows GRPO’s *RUR* decaying sharply over time — from above 90% to below 50% — regardless of model size or type; on Qwen2.5-Math-7B it plunges to about 30% after roughly 200 steps. This confirms that GRPO’s reliance on current-step advantage standardization produces a growing fraction of zero gradients, so its exploration becomes markedly inefficient late in training. DCPO’s smooth advantage standardization removes this decay: after the first epoch its *RUR* stabilizes around 70% and keeps improving with further training.

It is worth understanding why *RUR* settles near 70% rather than 100%, because doing so clarifies both a limitation and a hidden benefit of the SAS mechanism. *RUR* exceeds 90% in the first few steps but holds around 70% from the second epoch onward.

First, the early spike above 90% is largely an artifact: the untrained model often violates the required format, incurring negative rewards that inflate *RUR*. That models quickly learn fixed answer patterns argues for shifting emphasis toward exploring reasoning ability later in training. Second, two factors keep *RUR* near 70% rather than 100%, and both are benign: (i) nearly half of these problems are easy enough that the model solves them consistently, so excluding them from updates is justified — the model’s core capabilities need no further refinement there; and (ii) the remaining consistently-wrong instances arise either because some problems are too hard for the current model even with a correct label, or because the DAPO-Math-17K dataset contains items whose labels were converted to mislabeled integers. In the latter case, discarding zero-advantage responses effectively filters noisy or mislabeled data, so the residual gap is a feature of SAS rather than a failure of it.

Because DAPO discards responses with identical labels, a direct *RUR* comparison is not meaningful; we instead measure its efficiency by the generated responses and wall-clock time needed to complete the same number of update steps. To offset the low sampling efficiency induced by GRPO’s low *RUR*, we set DAPO’s generation batch size to $1\text{--}4\times$ the training batch size and adjust it dynamically. The result is stark: across the four models, DAPO needs $3\text{--}5\times$ more generated responses than DCPO for the same number of parameter updates, at least doubling training GPU hours without matching DCPO’s accuracy. DAPO’s data utilization also degrades with scale — on Qwen2.5-14B the fraction of used responses falls below 30% after about 300 steps — so its dynamic sampling both wastes compute and forgoes valuable diversity exploration.

Taken together, the *RUR* evidence substantiates our second and fourth contributions: DCPO advances clearly over both GRPO and DAPO by raising response utilization and training efficiency, using sampled data more fully to widen diversity exploration while improving performance.

5.4 ABLATION STUDIES

This ablation isolates the individual value of our three method contributions — OTM, SAS, and DAC — on Qwen2.5-Math-7B, using **Avg@32** to expose the robustness and stability of the learned policy. For fairness, each experiment changes exactly one component of the GRPO baseline while holding all else fixed and removing the KL divergence term to align with DAPO, GSPO, and the full DCPO. Figure 3 plots the learning curves from 20 to 400 steps.

The overall picture is clear: each of the three contributions, applied alone, already lifts the model above GRPO and up to or past DAPO and GSPO, and late in training the baselines plateau while DCPO — both per component and in full — keeps rising. In other words, every contribution adds a distinct benefit and the three compose into cumulative, stable gains. We detail each below.

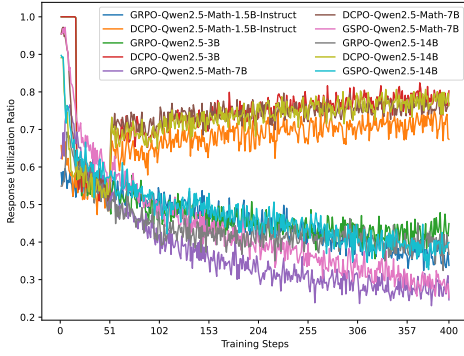


Figure 2: *RUR* progression during training: GRPO decays while DCPO holds a high plateau.

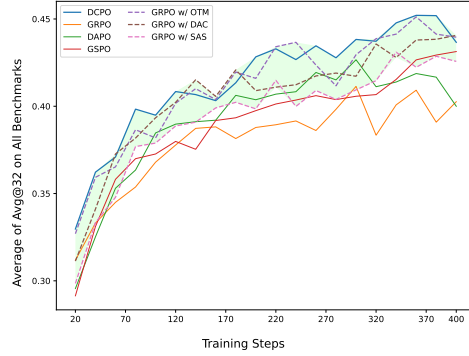


Figure 3: Ablation of DCPO’s three contributions (OTM, SAS, DAC), measured by average Avg@32 on Qwen2.5-Math-7B.

(i) **Only-Token-Mean loss (OTM)** — Contribution 3: aggregating the loss within a single response rather than across the whole batch already outperforms baselines, which suffer from lower data utilization and fluctuating performance. (ii) **Smooth Advantage Standardization (SAS)** — Contribution 2: replacing the stepwise advantage with the cumulative smoothed advantage markedly improves GRPO and approaches DAPO, showing that aggregating reward statistics across steps stabilizes optimization and raises final performance. (iii) **Dynamic Adaptive Clipping (DAC)** — Contribution 1: replacing GRPO’s fixed clipping boundary with the probability-aware adaptive boundary yields the largest single-component gain; DAC alone surpasses GRPO, DAPO, and GSPO and is more stable than OTM alone. (iv) **Full DCPO (OTM + SAS + DAC)** — Contribution 4 in miniature: combining the three mechanisms gives the best and most stable results, clearly beating GRPO, DAPO, and GSPO, and demonstrating that each design choice contributes uniquely to data-efficient, stable RL.

In summary, the ablation confirms that all three method contributions are individually beneficial and jointly synergistic, validating that DCPO’s mechanisms improve both data efficiency and stability in reinforcement learning for LLMs.

6 CONCLUSION

This work set out to fix two key limitations of existing policy optimization methods — restricted token-level exploration from fixed clipping in low-probability regions, and low sample efficiency from current-step reward standardization — and it does so through **Dynamic Clipping Policy Optimization (DCPO)**. We recap our contributions in the order they resolve those limitations. **First**, a **dynamically adaptive token-level clipping** mechanism (DAC) sets bounds from the old policy probabilities, unlocking efficient exploration of rare, low-probability tokens. **Second**, a **cumulative smooth advantage standardization** (SAS) aggregates the reward distribution across optimization steps, damping the reward randomness of high-entropy sampling. **Third**, the accompanying Only-Token-Mean loss preserves the relative advantage structure across responses. **Fourth**, extensive experiments on four benchmarks and four models confirm the payoff: DCPO-7B reaches an Avg@32 of 38.8 on the challenging AIME24 benchmark, outperforming GRPO (32.1) and DAPO (31.6), while DCPO raises response utilization by 28% over GRPO, doubles training efficiency over DAPO, and cuts the token clipping ratio by an order of magnitude. Together these contributions establish DCPO as a way to substantially improve data utilization and diversity exploration in RLVR. Future work will extend DCPO to other domains, such as code generation and semantic reasoning.

REFERENCES

Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Naveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*, 2025.

- DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jiawei Wang, Jin Chen, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, Junxiao Song, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Litong Wang, Liyue Zhang, Meng Li, Miaojuan Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qinyu Chen, Qiushi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runxin Xu, Ruoyu Zhang, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaoqing Wu, Shengfeng Ye, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Shuting Pan, T. Wang, Tao Yun, Tian Pei, Tianyu Sun, W. L. Xiao, Wangding Zeng, Wanbiao Zhao, Wei An, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, X. Q. Li, Xiangyue Jin, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaojin Shen, Xiaokang Chen, Xiaokang Zhang, Xiaosha Chen, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingkai Yu, Xinnan Song, Xinxia Shan, Xinyi Zhou, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Yang Zhang, Yanhong Xu, Yanhong Xu, Yanping Huang, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Li, Yaohui Wang, Yi Yu, Yi Zheng, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Ying Tang, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yu Wu, Yuan Ou, Yuchen Zhu, Yudian Wang, Yue Gong, Yuheng Zou, Yujia He, Yukun Zha, Yunfan Xiong, Yunxian Ma, Yuting Yan, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Z. F. Wu, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhen Huang, Zhen Zhang, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhipeng Xu, Zhiyu Wu, Zhongyu Zhang, Zhuoshu Li, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Ziyi Gao, and Zizheng Pan. Deepseek-v3 technical report, 2024. URL <https://arxiv.org/abs/2412.19437>.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021a.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *Sort*, 2(4): 0–6, 2021b.
- Jia Li, Edward Beeching, Lewis Tunstall, Ben Lipkin, Roman Soletskyi, Shengyi Huang, Kashif Rasul, Longhui Yu, Albert Q Jiang, Ziju Shen, et al. Numinamath: The largest public dataset in ai4maths with 860k pairs of competition math problems and solutions. *Hugging Face repository*, 13:9, 2024.
- Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding r1-zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025.
- MAA. American invitational mathematics examination (AIME). Mathematics Competition Series, 2024. URL <https://maa.org/math-competitions/aime>.
- OpenAI. Introducing openai o3 and o4-mini. 2025. URL <https://openai.com/index/introducing-o3-and-o4-mini/>.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.

- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. *arXiv preprint arXiv:2409.19256*, 2024.
- Shenzhi Wang, Le Yu, Chang Gao, Chujie Zheng, Shixuan Liu, Rui Lu, Kai Dang, Xionghui Chen, Jianxin Yang, Zhenru Zhang, et al. Beyond the 80/20 rule: High-entropy minority tokens drive effective reinforcement learning for llm reasoning. *arXiv preprint arXiv:2506.01939*, 2025.
- An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024a.
- An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, et al. Qwen2.5-math technical report: Toward mathematical expert model via self-improvement. *arXiv preprint arXiv:2409.12122*, 2024b.
- Deheng Ye, Zhao Liu, Mingfei Sun, Bei Shi, Peilin Zhao, Hao Wu, Hongsheng Yu, Shaojie Yang, Xipeng Wu, Qingwei Guo, et al. Mastering complex control in moba games with deep reinforcement learning. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pp. 6672–6679, 2020.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.
- Yufeng Yuan, Qiyang Yu, Xiaochen Zuo, Ruofei Zhu, Wenyuan Xu, Jiaze Chen, Chengyi Wang, TianTian Fan, Zhengyin Du, Xiangpeng Wei, et al. Vapo: Efficient and reliable reinforcement learning for advanced reasoning tasks. *arXiv preprint arXiv:2504.05118*, 2025.
- Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in llms beyond the base model? *arXiv preprint arXiv:2504.13837*, 2025.
- Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, et al. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*, 2025.

A APPENDIX

A.1 MAIN SYMBOL DEFINITIONS

G	Number of responses generated per prompt at each step.
θ	Parameters of the actor model.
$\pi_{\theta}(o_i q)$	Probability of response o_i given prompt q under parameters θ .
$\pi_{\theta_{old}}(o_i q)$	Probability of response o_i given prompt q under previous parameters θ_{old} .
π_{ref}	Reference policy for KL divergence computation.
β	Coefficient for KL divergence term (controls policy update vs. constraint trade-off).
$r_{i,t}(\theta)$	Probability ratio $\frac{\pi_{\theta}(o_i^t q, o_i^{1:t-1})}{\pi_{\theta_{old}}(o_i^t q, o_i^{1:t-1})}$ for token t in response o_i .
$\hat{A}_{i,t}$	Standardized advantage estimate for token t in response o_i .
$ o_i $	The length of the response in tokens.
$\mathbb{D}_{KL}(\pi_{\theta} \pi_{ref})$	KL divergence between current and reference policies.
ε	Probability ratio clipping threshold.
ε_{low} and ε_{high}	Lower/upper thresholds for probability ratio clipping.
M	Maximum allowed response length.
$is_equivalent(a, o_i)$	Binary function indicating whether response o_i matches answer a .
μ_{new}^i	Mean reward across G responses generated at step i (0 if $i = 0$).
σ_{new}^i	Reward variance across G responses at step i (0 if $i = 0$).
μ_{old}^i	Mean reward across $G \cdot (i - 1)$ responses from steps 1 to $i - 1$ (0 if $i \leq 1$).
σ_{old}^i	Reward variance across $G \cdot (i - 1)$ responses from steps 1 to $i - 1$ (0 if $i \leq 1$).
μ_{total}^i	Mean reward across $G \cdot i$ responses from steps 1 to i (0 if $i = 0$).
σ_{total}^i	Reward variance across $G \cdot i$ responses from steps 1 to i (0 if $i = 0$).

A.2 PRELIMINARY

A.3 ADVANTAGE CALCULATION

To make our contributions self-contained, this appendix reviews the baselines DCPO builds on. In previous works, including GRPO and DAPO, the advantage $\hat{A}_{j,t}^i$ for token t in response j is obtained by standardizing the reward R_j^i against the mean μ^i and standard deviation σ^i of the rewards of the G responses generated at the i -th step, as shown in Equation (11).

$$\hat{A}_{j,t}^i = \frac{(R_j^i - \mu^i)}{\sigma^i} \quad (11)$$

When the rewards for the same prompt are identical, responses with zero advantage do not contribute to the model update, resulting in wasted responses — the very inefficiency our second contribution removes.

A.3.1 GRPO

Shao et al. (2024) proposed the **Group Relative Policy Optimization (GRPO)**, which first samples G responses for each query, assigns rewards R through a rule-based reward function and estimates token-level advantages as in Equation (11). Finally, the model parameters are updated using Equation (12), referred to as **Sequence Level Mean loss (SLM)** as:

$$\mathcal{T}_{\text{GRPO}}(\theta) = \frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \min \left(r_{i,t}(\theta) \hat{A}_{i,t}, \text{clip}(r_{i,t}(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_{i,t} \right) - \beta \mathbb{D}_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}}) \quad (12)$$

The first term averages token-level advantages over all tokens in a response and then over all G responses in the batch; the $\mathbb{D}_{\text{KL}}$ penalty keeps the policy close to a reference (e.g., a pre-trained model); and the fixed clipping threshold ε caps updates by limiting the change in token probability ratios. This double averaging, though widely adopted, dilutes per-response importance — exactly the bias our third contribution (OTM) is designed to remove.

A.3.2 DAPO

Similarly, Yu et al. (2025) propose **Dynamic sAmpling Policy Optimization (DAPO)**, observing that in GRPO longer sequences exert disproportionate influence on the gradient update. DAPO standardizes token counts across the entire batch, giving the **Token Level Mean loss (TLM)** of Equation (13).

$$\mathcal{T}_{\text{DAPO}}(\theta) = \frac{1}{\sum_{i=1}^G |o_i|} \sum_{i=1}^G \sum_{t=1}^{|o_i|} \min \left(r_{i,t}(\theta) \hat{A}_{i,t}, \text{clip}(r_{i,t}(\theta), 1 - \varepsilon_{\text{low}}, 1 + \varepsilon_{\text{high}}) \hat{A}_{i,t} \right) \quad (13)$$

subject to $0 < |\{o_i \mid \text{is_equivalent}(a, o_i)\}| < G$. DAPO discards every prompt whose responses all share the same reward and regenerates responses to refill the batch, which is the source of the severe data inefficiency our second contribution avoids.

A.3.3 GSPO

Zheng et al. (2025) propose **Group Sequence Policy Optimization (GSPO)**, arguing that token-level clipping injects high-variance noise into gradient estimates, noise that accumulates over long sequences and is amplified by clipping itself. GSPO replaces token-wise with sequence-wise clipping for a more stable signal — a different trade-off from our first contribution, which keeps token-level updates but makes their bounds probability-aware.

$$\mathcal{T}_{\text{GSPO}}(\theta) = \frac{1}{G} \sum_{i=1}^G \min \left(s_i(\theta) \hat{A}_i, \text{clip}(s_i(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_i \right) \quad (14)$$

$$\text{where } s_i(\theta) = \exp \left(\frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \log \left(\frac{\pi_{\theta}(o_{j,t}|q)}{\pi_{\theta_{\text{old}}}(o_{j,t}|q)} \right) \right)$$

In practice GSPO discards more than 10% of responses that carry non-zero advantages, wasting data and hurting training efficiency. GRPO removes sequence-level high-variance responses yet still retains many token-level high-entropy responses, which destabilizes training. DCPO instead keeps these informative tokens through its dynamic-adaptive bounds.

A.4 REWARD CALCULATION

For a verifiable and interpretable supervision signal, we adopt a rule-based reward model that scores each response on answer correctness and format compliance, as in Equation (15): +1 when both format and answer are correct, 0 when the format is correct but the answer is wrong, and -1 when the format is incorrect.

$$R_j^i = \begin{cases} 1, & \text{correct format and correct answer} \\ 0, & \text{correct format but incorrect answer} \\ -1, & \text{incorrect format} \end{cases} \quad (15)$$

where j is the index of the response group for a given problem in the i -th training step, and the reward is calculated based on the format and correctness of the response.

A.5 HYPOTHETICAL REASONING PROCESS FOR DYNAMIC-ADAPTIVE CLIPPING BOUNDS

This appendix gives the full derivation underpinning our first contribution, the dynamic-adaptive clipping bounds. Clipping the probability ratio is a crucial technique in reinforcement learning: it stabilizes policy optimization and prevents large, destabilizing parameter updates, and it matters most when importance sampling is used to estimate expected values under a new policy from samples generated by an old one.

The expected value of a function $f(x)$ under the new probability $p(x)$ can be rewritten as an expectation under the old probability $q(x)$ by importance sampling weight, as shown in equation 16:

$$\int f(x)p(x) dx = \int f(x) \frac{p(x)}{q(x)} q(x) dx = \mathbb{E}_{x \sim q} \left[f(x) \frac{p(x)}{q(x)} \right] \quad (16)$$

Although this estimator is unbiased, its variance can be significantly inflated, which is a common challenge in importance sampling. The true variance of the function $f(x)$ under the new policy is given by Equation (17), whereas the variance of the importance-sampled estimator under the old policy is given by Equation (18):

$$\text{Var}_{x \sim p}[f(x)] = \mathbb{E}_{x \sim p} [f(x)^2] - (\mathbb{E}_{x \sim p}[f(x)])^2 \quad (17)$$

$$\begin{aligned} \text{Var}_{x \sim q} \left[f(x) \frac{p(x)}{q(x)} \right] &= \mathbb{E}_{x \sim q} \left[\left(f(x) \frac{p(x)}{q(x)} \right)^2 \right] - \left(\mathbb{E}_{x \sim q} \left[f(x) \frac{p(x)}{q(x)} \right] \right)^2 \\ &= \mathbb{E}_{x \sim p} \left[f(x)^2 \frac{p(x)}{q(x)} \right] - (\mathbb{E}_{x \sim p}[f(x)])^2 \end{aligned} \quad (18)$$

The difference between the importance-sampled variance and the true variance highlights the potential for inflation, as shown in Equation (19):

$$\text{Var}_{x \sim q} \left[f(x) \frac{p(x)}{q(x)} \right] - \text{Var}_{x \sim p} [f(x)] = \mathbb{E}_{x \sim p} \left[f(x)^2 \left(\frac{p(x)}{q(x)} - 1 \right) \right] = \int f(x)^2 \left(\frac{p(x)}{q(x)} - 1 \right) p(x) dx \quad (19)$$

To mitigate this difference, many prior works, including PPO Schulman et al. (2017) and RLVR methods (DeepSeek-AI et al., 2024; Liu et al., 2025), employ a fixed symmetric clipping bound on the probability ratio $r(x) = \frac{p(x)}{q(x)}$, as defined in Equation (20):

$$|r(x) - 1| \leq \epsilon \quad (20)$$

The problem, again, is that a fixed clipping bound constrains exploration, most acutely where the old policy assigns low probability mass: there the absolute bound is severely limited and meaningful updates are suppressed. DAPO’s asymmetric bounds only partially help, since the constraint remains non-adaptive.

Recent studies such as Yue et al. (2025) further suggest that the reasoning paths RLVR exploits already lie within the base model’s sampling distribution, consistent with the minimal parameter shift — and small clipped fraction — seen during off-policy batch updates. This stability is what makes our first contribution well-founded: we constrain the probability ratio $r(x)$ through a dynamic-adaptive mechanism that folds the probability into the restriction, as in Equation (21), replacing the conventional fixed clipping of Equation (20).

$$\left| \left(\frac{p(x)}{q(x)} - 1 \right) p(x) \right| \leq \epsilon \quad (21)$$

Expanding the substitution of $p(x) = r(x)q(x)$ into the inequality of a previous equation yields the expression in Equation (22).

$$-\epsilon \leq (r(x) - 1)r(x) * q(x) \leq \epsilon \quad (22)$$

For the resulting clipping ratio to be valid, it must respect the basic constraints on probabilities and their ratios. As stated in Equation (23), $p(x)$ and $q(x)$ lie in $[0, 1]$, so the ratio $r(x)$ and the hyper-parameter ϵ are both non-negative.

$$\begin{cases} 0 \leq p(x) \leq 1 \\ 0 \leq q(x) \leq 1 \\ 0 \leq r(x) \\ 0 \leq \epsilon \end{cases} \quad (23)$$

To ensure the validity and practical applicability of the clipping bound derived from Equation (22), we define the necessary conditions for its bounds as outlined in Equation (24).

$$\begin{cases} low_{ge}(q(x), \epsilon) = 0.5 - \frac{1}{2}\sqrt{1 + \frac{4\epsilon}{q(x)}} \\ high_{le}(q(x), \epsilon) = 0.5 + \frac{1}{2}\sqrt{1 + \frac{4\epsilon}{q(x)}} \\ low_{le}(q(x), \epsilon) = 0.5 - \frac{1}{2}\sqrt{1 - \frac{4\epsilon}{q(x)}}, & \text{S.t. } q(x)^2 - 4q(x) * \epsilon \geq 0 \\ high_{ge}(q(x), \epsilon) = 0.5 + \frac{1}{2}\sqrt{1 - \frac{4\epsilon}{q(x)}}, & \text{S.t. } q(x)^2 - 4q(x) * \epsilon \geq 0 \end{cases} \quad (24)$$

As we can get from Equation (22) to Equation (24), the probability ratio $r(x)$ should be met by the following conditions:

$$\begin{cases} low_{ge}(q(x), \epsilon) \leq 0 \leq r(x) \leq high_{le}(q(x), \epsilon), & \text{when } \text{Var}_{x \sim q} \left[f(x) \frac{p(x)}{q(x)} \right] \geq \text{Var}_{x \sim p} [f(x)] \\ r(x) \leq low_{le}(q(x), \epsilon) \cup high_{ge}(q(x), \epsilon) \leq r(x), & \text{otherwise} \end{cases} \quad (25)$$

To choose among these, we take the overlapping region centered on a ratio of 1, which both minimizes the discrepancy between the two conditions and stays consistent with GRPO's conventional fixed bound; the resulting validity conditions are:

$$\begin{aligned} high_{ge}(q(x), \epsilon) \leq r(x) \leq high_{le}(q(x), \epsilon) \\ \text{S.t. } q(x)^2 - 4q(x) * \epsilon \geq 0 \end{aligned} \quad (26)$$

For low_{le} and $high_{ge}$, to ensure that the square number is not less than zero, we use the method of comparing with 0 and taking the maximum value of the approximation like $\max(1 - \frac{4\epsilon}{q(x)}, 0)$. Finally, we can get the following conditions, as **dynamic-adaptive clipping bounds for importance sampling**:

$$0.5 + \frac{1}{2}\sqrt{\max\left(1 - \frac{4\epsilon_{low}}{q(x)}, 0\right)} \leq r(x) \leq 0.5 + \frac{1}{2}\sqrt{1 + \frac{4\epsilon_{high}}{q(x)}} \quad (27)$$

For the upper and lower bounds in Inequality 27, we use distinct thresholds ϵ_{high} and ϵ_{low} so that our adaptive bounds coincide with GRPO's fixed bounds at specific reference points — the design that makes our first contribution a strict generalization rather than a departure.

A.6 ANALYZING THE DIFFERENCES BETWEEN DYNAMIC-ADAPTIVE CLIPPING AND FIXED CLIPPING

This subsection quantifies precisely how our first contribution differs from fixed clipping. To align our dynamic-adaptive bounds with the fixed symmetric bounds at the boundary points $(\frac{1}{1+\epsilon}, 1)$ and $(1, 1 - \epsilon)$, as shown in Figure 4d, we solve Equation (28) for the specific values of ϵ_{low} and ϵ_{high} .

$$\begin{cases} q(x) = \frac{1}{1+\epsilon} \\ 1 = 0.5 + \frac{1}{2} \sqrt{1 + \frac{4\epsilon_{high}}{q(x)}} \end{cases} \quad (28)$$

$$\begin{cases} q(x) = 1 \\ 1 - \epsilon = 0.5 + \frac{1}{2} \sqrt{\max(1 - \frac{4\epsilon_{low}}{q(x)}, 0)} \end{cases}$$

With the usual GRPO setting $\epsilon = 0.2$, this yields approximately $\epsilon_{low} = 0.16$ and $\epsilon_{high} = 0.2$. Note that here $\epsilon_{low} \neq \epsilon_{high}$ serves a different purpose than DAPO’s asymmetric fixed clipping: it enforces alignment at the reference points, not a manually widened upper bound.

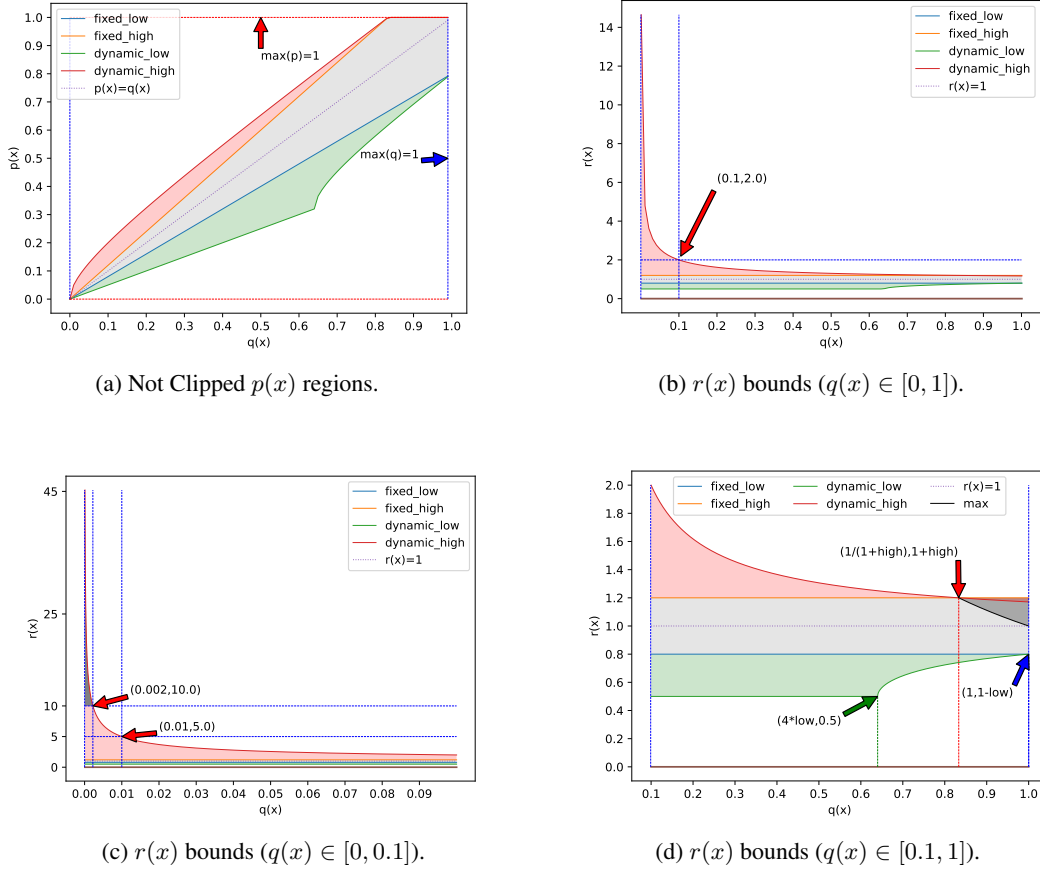


Figure 4: Clipping-bound comparison illustrating our first contribution: fixed clipping ($\epsilon = 0.2$) vs. dynamic-adaptive clipping ($\epsilon_{low} = 0.16$, $\epsilon_{high} = 0.2$), which widens the admissible region for rare, low-probability tokens.

Dynamic versus fixed clipping: Figure 4 contrasts GRPO’s fixed scheme ($[1 - \epsilon, 1 + \epsilon]$) with our probability-aware adaptive scheme. For a token with old probability $q(x)$, the fixed bound has a constant relative width 2ϵ , so its absolute exploration space shrinks linearly as $q(x)$ falls (Figure 4a). Our adaptive width is instead inversely proportional to $q(x)$, keeping the exploration space roughly constant across the probability range — the core behavioral advantage of this contribution.

Effect of decreasing $q(x)$: When $q(x)$ is small, the fixed interval $[1 - \epsilon, 1 + \epsilon]$ is independent of $q(x)$ and does nothing to help explore low-probability tokens, whereas our adaptive bound expands rapidly (Figure 4b), granting a larger relative space to rare tokens — precisely the behavior efficient RL requires.

Behavior for extremely low probabilities: Figure 4c plots the clipping interval for $q(x) \in [0, 0.1]$. As $q(x) \rightarrow 0$, our upper bound for $r(x)$ grows like an inverse square root while the fixed bound stays

constant. To keep this freedom safe, and inspired by the dual-clipping strategy (Ye et al., 2020), we enforce a hard ceiling $r_{\max} = 10$ against instabilities (vanishing or exploding gradients). The gray region in Figure 4c marks approximately $q(x) \leq 2 \times 10^{-3}$, where the adaptive upper limit is capped at $r_{\max}$.

Upper and lower bounds: For the lower bound, when $q(x) \leq 4\varepsilon$ our scheme yields $dynamic_{low} = 0.5$ while fixed clipping keeps the constant $fixed_{low} = 1 - \varepsilon = 0.8$; beyond this threshold our lower bound smoothly contracts toward the fixed value. Symmetrically, our upper bound exceeds the fixed bound whenever $q(x) < \frac{1}{1+\varepsilon_{high}}$ and merges with it for larger $q(x)$. The adaptive interval $[dynamic_{low}(q(x), \epsilon_{low}), dynamic_{high}(q(x), \epsilon_{high})]$ therefore widens the admissible region for rare tokens while preserving the tight interval for common tokens (Figure 4d) — exactly the selectivity claimed for our first contribution.

Positive versus negative advantage: The loss in Equation (8) has both positive- and negative-advantage terms. Under the minimum operation, a positive advantage clips $r(x)$ to $[0, dynamic_{high}(q(x), \epsilon_{high})]$ and a negative advantage to $[dynamic_{low}(q(x), \epsilon_{low}), \infty)$. Since an excessively large $r(x)$ can destabilize training, we cap the ratio at $r_{\max} = 10$ for both cases; this precaution, following Ye et al. (2020), keeps the adaptive limits from producing runaway importance weights.

In short, for both positive and negative advantages, dynamic-adaptive clipping gives the model more room to explore diversity precisely when rare tokens have low probability — the benefit that our TCR and accuracy results then confirm empirically.

A.7 THE DEFINITIONS OF THE ADVANTAGE STANDARDIZATION

This subsection formalizes the statistics that our second contribution (SAS) relies on. Let i denote the i -th iteration in which prompts generate responses and update the parameters. We define the following statistics for reward normalization.

For the mean reward of the responses, we use the group of functions in Equation (29):

$$\begin{cases} \mu_{new}^i = \frac{1}{G} \sum_{j=1}^G R_j^i \\ \mu_{old}^i = \frac{1}{G*(i-1)} \sum_{k=1}^{i-1} \sum_{j=1}^G R_j^k \\ \mu_{total}^i = \frac{1}{G*i} \sum_{k=1}^i \sum_{j=1}^G R_j^k \\ \quad = \frac{1}{i} (\mu_{new}^i + (i-1) \mu_{old}^i) \end{cases} \quad (29)$$

For the reward variance of the responses, we use the group of functions in Equation (30):

$$\begin{cases} \sigma_{new}^i = \sqrt{\frac{1}{G} \sum_{j=1}^G (R_j^i - \mu_{new}^i)^2} \\ \sigma_{old}^i = \sqrt{\frac{1}{G*(i-1)} \sum_{k=1}^{i-1} \sum_{j=1}^G (R_j^k - \mu_{old}^i)^2} \\ \sigma_{total}^i = \sqrt{\frac{1}{G*i} \sum_{k=1}^i \sum_{j=1}^G (R_j^k - \mu_{total}^i)^2} \\ \quad = \sqrt{\frac{1}{i} \left(\sigma_{new}^i{}^2 + (i-1) \sigma_{old}^i{}^2 + \frac{i-1}{i} (\mu_{old}^i - \mu_{new}^i)^2 \right)} \end{cases} \quad (30)$$

To keep SAS cheap in practice, we compute σ_{total}^i and μ_{total}^i in Equation (30) and Equation (29) through the equivalent recurrences over $\mu_{new}^i, \sigma_{new}^i, \mu_{old}^i$ and σ_{old}^i rather than re-scanning the original rewards, so the contribution adds negligible overhead.

A.8 THE TEMPLATE FOR EXPERIMENTS

For reproducibility of the experiments supporting our fourth contribution, all runs in this paper use the following template, based on the Qwen-Math template (Yang et al., 2024b).

Qwen-Math Template

```
<|im_start|>system
Please reason step by step, and put your final answer within \boxed{ }. <|im_end|>
<|im_start|>user
{problem}
<|im_end|>
<|im_start|>assistant
```

A.9 TRAINING SETTING

For fair comparison, each method uses its own recommended clipping configuration: GRPO uses a symmetric threshold $\epsilon = 0.2$; DAPO uses lower/upper thresholds $(0.2, 0.28)$ with an additional 512-token soft-punish cache and a maximum probability ratio of 10 for negative advantage; GSPO uses lower/upper thresholds $(3e - 4, 4e - 4)$; and DCPO uses low/high thresholds $(0.16, 0.2)$. Crucially, DCPO’s thresholds preserve the same clipping points $(q(x), r(x)) = (\frac{1}{1+\epsilon}, 1 + \epsilon)$ and $(q(x), r(x)) = (1, 1 - \epsilon)$ as GRPO (Figure 4d), and cap the maximum probability ratio at 10 for *both* positive and negative advantages — unlike DAPO — so any gains reflect our contributions rather than a looser bound.

We trained for 400 steps with a generation batch size of 512 and a parameter-update mini-batch size of 32. With temperature 1 and top_p 1, we generated $G=16$ responses per problem, capping the prompt at 1,024 tokens and the response at 3,072 tokens — far below the 20k used in the original DAPO.

We adapted our codebase from Verl (Sheng et al., 2024) on $32 \times H20$ GPUs, matching DAPO’s hyperparameters unless stated otherwise. For loss computation we kept GRPO’s and DAPO’s exact $\frac{1}{G}$ or $\sum_{i=1}^G \frac{1}{|o_i|}$ weighting rather than Verl’s approximate weighting, ensuring compatibility across microbatch sizes.

A.10 THE DETAILED PERFORMANCE OF MODELS

Avg@1 Trend Figure 5 tracks the average Avg@1 metric — the mean greedy-decoding accuracy over MATH500, AMC23, AIME24, and AIME25 — over training. The trajectory reinforces our accuracy contribution: DCPO consistently outperforms GRPO and DAPO on all four models, and beats GSPO on the two models where it was evaluated. It is also steadier, avoiding both GRPO’s sharp drops and DAPO’s large fluctuations, which indicates that DCPO elicits stronger reasoning during training and leaves more headroom for further gains.

Avg@32 Trend Figure 6 plots Avg@32 for DCPO against GRPO, DAPO and (where available) GSPO across model sizes on AMC23, AIME24, and AIME25, and the contrast is decisive. GRPO fluctuates heavily and occasionally collapses — Qwen2.5-14B under GRPO drops sharply after roughly 170 steps. DCPO instead trends consistently upward with minimal fluctuation across every model from 1.5B to 14B, grows fastest even early in training, and matches or beats DAPO while saving half the training time and resources. In sum, this evidence ties our first two contributions to outcomes: dynamic-adaptive clipping and smooth advantage standardization jointly improve exploration at both the response level and the label level of diversity.

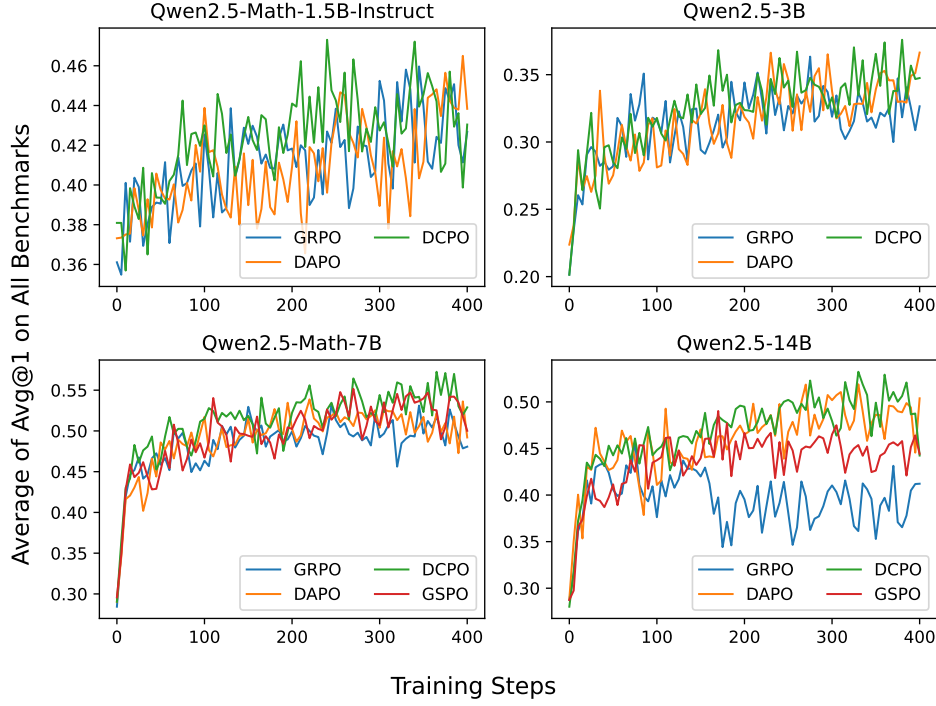


Figure 5: Avg@1 performance across benchmarks, providing training-time support for our accuracy contribution.

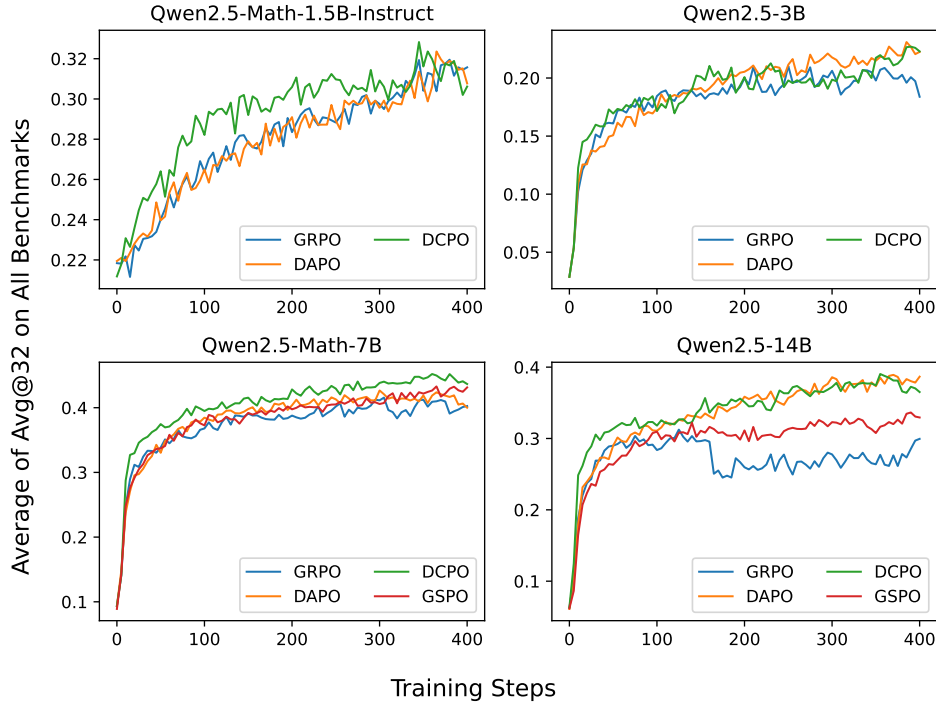


Figure 6: Avg@32 performance across benchmarks: DCPO’s robustness and stability under sampling.

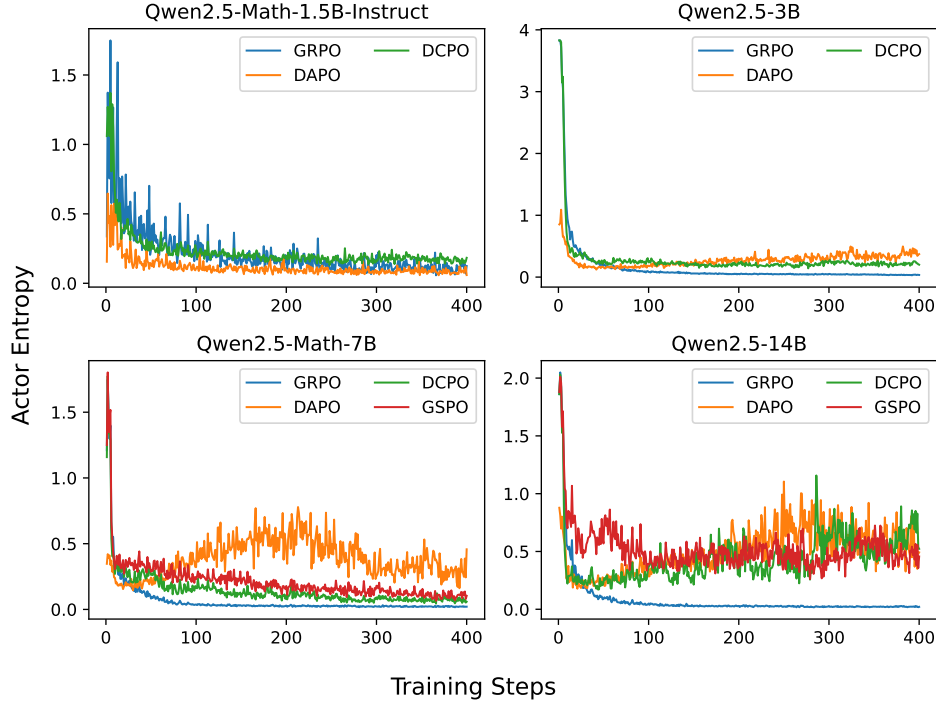


Figure 7: Model entropy during training: DCPO stays in a moderate, well-balanced band, unlike GRPO’s collapse and DAPO’s high-variance regime.

Entropy Trend Entropy dynamics offer a mechanistic explanation for why our contributions help. Figure 7 shows GRPO’s entropy decaying sharply after warm-up and settling into a flat, low-entropy range — an entropy collapse that drains policy randomness, shrinks response diversity, and stalls further optimization.

DAPO and DCPO behave in a model-dependent way. On the Instruct model (Qwen2.5-Math-1.5B), DAPO, DCPO, and GRPO follow similar declining trajectories, but on the larger base models (Qwen2.5-3B, Qwen2.5-Math-7B, Qwen2.5-14B) they diverge. DAPO holds the highest entropy with a clear upward trend and large fluctuations. On Qwen2.5-Math-7B, GSPO’s entropy and average performance both resemble DCPO’s; on Qwen2.5-14B, however, GSPO drifts downward with fluctuations and performs poorly. DCPO stays at a moderate level, its curve markedly smoother and confined to a band between GRPO’s low-entropy floor and DAPO’s high-variance ceiling.

This points to a U-shaped relationship between entropy and performance: too little entropy starves exploration and causes premature convergence, while too much or too unstable entropy breaks training stability. By keeping entropy in a moderate, well-balanced band — increasingly so as model capacity grows — DCPO promotes both convergence and sufficient exploration. This entropy regulation is a natural consequence of our clipping and smoothing contributions and helps explain DCPO’s stronger, more robust reasoning across model sizes.